# EN4554 Assignment 2 — Convolutional Neural Networks

Your Name | Index: Your Index Number | GitHub: https://github.com/your-username/en4554-a2

19 October 2026 | PyTorch 2.11, Google Colab Tesla T4

## Setup

CIFAR-10 training images were split into 45,000 train / 5,000 validation images (fixed random permutation, seed 0; validation classes are balanced, 467–532 per class). Per-channel mean $(0.4912, 0.4820, 0.4464)$ and std $(0.2472, 0.2436, 0.2617)$ were computed on the 45k training images only and used for all splits. All seeds (Python, NumPy, PyTorch, CUDA) are reset before every run. The whole dataset is held on the GPU and augmentation (random flip, random $32 \times 32$ crop of a 4-pixel zero-padded image, cutout, mixup) is done batch-wise on the GPU. The test set is loaded only once, at the end of Task 4. Every experiment is cached to JSON so that a Colab disconnect does not lose finished runs. All results are from a single seed, so differences of about $\pm 0.5$ percentage points (pp) in validation accuracy are within noise (binomial standard error on 5k images $\approx 0.45$ pp at 89%).

## 1 Task 1 — Backpropagation

**(a) Implementation.** Each layer caches what it needs in `forward` and returns $\partial L/\partial(\text{input})$ from `backward`; no autograd is used. The MLP is a list of layers, and its `backward` runs them in reverse.

```
class Linear:
    def forward(self, x):
        self.x = x;  return x @ self.W + self.b
    def backward(self, g):                          # g = dL/d(out), shape (N, n_out)
        self.grads = [self.x.T @ g, g.sum(0)]       # dL/dW, dL/db
        return g @ self.W.T                         # dL/dx
class ReLU:
    def forward(self, x):  self.mask = x > 0;  return x * self.mask
    def backward(self, g): return g * self.mask
class SoftmaxCrossEntropy:
    def forward(self, z, y):
        z = z - z.max(1, keepdim=True).values               # numerical stability
        logp = z - z.exp().sum(1, keepdim=True).log()
        self.p, self.y = logp.exp(), y
        return -logp[torch.arange(len(y)), y].mean()
    def backward(self):                                     # (softmax(z) - onehot(y)) / N
        g = self.p.clone();  g[torch.arange(len(self.y)), self.y] -= 1
        return g / len(self.y)
```

**(b) Gradient check.** A 20–16–16–10 network (float64, batch of 8) was checked against autograd (the same forward pass rebuilt with `torch` ops) and against centred finite differences $(L(\theta + \varepsilon e_i) - L(\theta - \varepsilon e_i))/2\varepsilon$, $\varepsilon = 10^{-6}$, for *every* entry. The relative error is $\|a - b\|_2 / (\|a\|_2 + \|b\|_2)$.

**Table 1:** Relative error of the manual gradients. `torch.allclose` is `True` for all 28 comparisons.

| | ReLU network | | Sigmoid network | |
|---|---|---|---|---|
| Tensor | vs autograd | vs finite diff. | vs autograd | vs finite diff. |
| $W_1$ (20×16) | $5.9 \times 10^{-17}$ | $7.6 \times 10^{-10}$ | $1.1 \times 10^{-16}$ | $1.4 \times 10^{-8}$ |
| $b_1$ | $3.9 \times 10^{-17}$ | $4.3 \times 10^{-10}$ | $1.1 \times 10^{-16}$ | $1.2 \times 10^{-8}$ |
| $W_2$ (16×16) | $4.4 \times 10^{-17}$ | $5.2 \times 10^{-10}$ | $9.0 \times 10^{-17}$ | $5.7 \times 10^{-9}$ |
| $b_2$ | $6.6 \times 10^{-17}$ | $5.9 \times 10^{-10}$ | $1.1 \times 10^{-16}$ | $3.4 \times 10^{-9}$ |
| $W_3$ (16×10) | $3.4 \times 10^{-17}$ | $5.9 \times 10^{-10}$ | $7.4 \times 10^{-17}$ | $1.4 \times 10^{-9}$ |
| $b_3$ | $2.0 \times 10^{-17}$ | $1.1 \times 10^{-9}$ | $6.2 \times 10^{-17}$ | $7.3 \times 10^{-10}$ |
| input $x$ (8×20) | $7.6 \times 10^{-17}$ | $1.3 \times 10^{-9}$ | $1.3 \times 10^{-16}$ | $3.9 \times 10^{-8}$ |

*Interpretation.* The errors against autograd ($2 \times 10^{-17}$–$1.3 \times 10^{-16}$) are at float64 machine precision ($2.2 \times 10^{-16}$). The two methods compute the same expressions, only with different rounding, so the backward pass is exactly right. The finite-difference errors ($4 \times 10^{-10}$–$4 \times 10^{-8}$) are what a numerical derivative can achieve. They come from an $O(\varepsilon^2)$ truncation term plus an $O(\delta/\varepsilon)$ round-off term ($\delta \approx 10^{-16}$). The sigmoid network has somewhat larger errors because its loss has larger higher-order derivatives, while a ReLU network is piecewise linear in each layer's input. Both checks agree, which confirms the hand-derived gradients.

**(c) Training the MLP.** A 3072–512–256–10 ReLU MLP (He initialisation, 1.71M parameters) was trained with the manual layers using plain mini-batch SGD (batch 128, constant learning rate 0.02) for 20 epochs on flattened images.

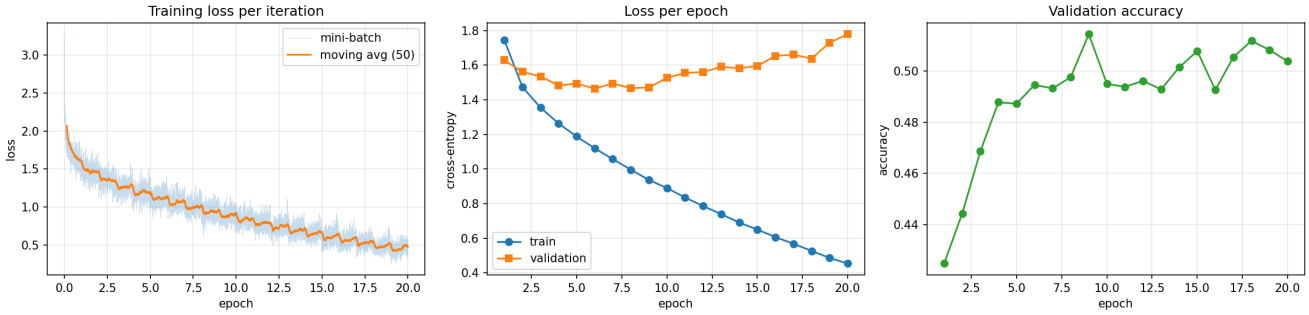
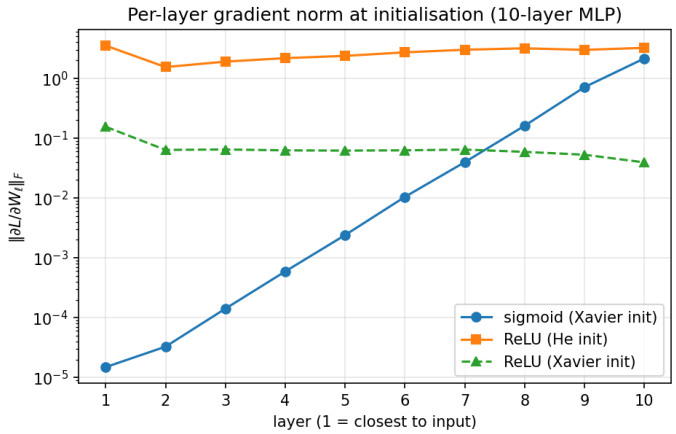



**Figure 1:** Manual MLP trained with SGD: per-iteration loss, train/validation loss per epoch, and validation accuracy.

*Interpretation.* The training loss falls steadily from 1.75 to 0.45, so the hand-written backward pass trains the network correctly. Validation loss reaches its minimum (1.46) at epoch 6 and then rises to 1.78, while validation accuracy plateaus at about 50% (best 51.4% at epoch 9, final 50.4%). From epoch ∼6 onwards the MLP is overfitting: it memorises the training images (train loss 0.45 against validation loss 1.78) without generalising better. The accuracy ceiling of about 50% is a property of the architecture. Flattening discards the 2-D layout of the image, and a fully connected layer has no weight sharing and no translation equivariance, so the same edge at two positions has to be learned twice. The CNNs below reach 68–76% in half as many epochs. The mini-batch noise and the epoch-to-epoch fluctuation of validation accuracy reflect the constant learning rate; early stopping near epoch 9 or a decaying learning rate would help.

**(d) Gradient norm at initialisation, 10-layer MLP.** The network is $3072 \to 9\times256 \to 10$, with one forward/backward pass on 512 images.




| | $\|g_1\|$ | $\|g_{10}\|$ | $\|g_1\|/\|g_{10}\|$ |
|---|---|---|---|
| sigmoid, Xavier | $1.5 \times 10^{-5}$ | 2.14 | $7.0 \times 10^{-6}$ |
| ReLU, He | 3.51 | 3.22 | 1.09 |
| ReLU, Xavier | 0.155 | 0.039 | 3.94 |

**Figure 2:** Per-layer $\|\partial L/\partial W_\ell\|_F$ at initialisation (log scale) and first/last layer ratios.

*Interpretation.* With sigmoid the gradient norm falls by more than five orders of magnitude from the last layer to the first, a nearly constant factor of $\approx 0.27$ per layer (a straight line on the log plot). In backpropagation each layer multiplies the signal by $W_\ell^\top \operatorname{diag}(\sigma'(z_\ell))$, and $\sigma'(z) = \sigma(1-\sigma) \le 0.25$. Xavier initialisation keeps the gain of $W$ near 1, so each layer contributes a factor of roughly 1/4. The result is *vanishing gradients*: the first layers receive almost no learning signal and would barely train. With ReLU and He initialisation the norms stay within a factor of about 2 across depth. ReLU's derivative is 1 on the active half of the units, and the He variance $2/n_{\text{in}}$ exactly compensates for the half that is zeroed, so the signal variance is preserved in both directions. The ReLU + Xavier control shows that the initialisation matters as well as the activation. With variance 1/256 the activations shrink by $\sqrt{2}$ per layer going forward and the back-propagated errors shrink by $\sqrt{2}$ per layer going backward. In $\partial L/\partial W_\ell = a_{\ell-1}^\top \delta_\ell$ the two effects cancel, which is why the curve is flat, but the overall scale is about $50\times$ smaller than with He initialisation.

# 2 Task 2 — Optimizers

**(a) Implementation and check.** Both optimizers subclass `torch.optim.Optimizer` and follow PyTorch's conventions: L2 weight decay is added to the gradient, and the first momentum buffer equals the first gradient. The core of the Adam step:

```
state['t'] += 1;  t = state['t']
state['m'].mul_(b1).add_(g, alpha=1 - b1)                # m = b1 m + (1-b1) g
state['v'].mul_(b2).addcmul_(g, g, value=1 - b2)         # v = b2 v + (1-b2) g^2
m_hat = state['m'] / (1 - b1 ** t)                       # bias correction
v_hat = state['v'] / (1 - b2 ** t)
p.addcdiv_(m_hat, v_hat.sqrt().add_(eps), value=-lr)     # p -= lr * m_hat / (sqrt(v_hat) + eps)
```

Two identical float64 networks (3072–64–10) were trained for 100 steps on the same CIFAR-10 batches, one with my optimizer and one with PyTorch's.

**Table 2:** My optimizers vs `torch.optim` after 100 steps.

| Optimizer | Hyper-parameters | max $\lvert\Delta\theta\rvert$ | max $\lvert\Delta L\rvert$ | `allclose` |
|---|---|---|---|---|
| SGD | lr 0.05 | 0 | 0 | True |
| SGD + momentum | lr 0.05, $\mu$ 0.9 | 0 | 0 | True |
| SGD + momentum + wd | lr 0.05, $\mu$ 0.9, wd $5\times10^{-4}$ | 0 | 0 | True |
| Adam | lr $10^{-3}$ | $4.2 \times 10^{-17}$ | $4.4 \times 10^{-16}$ | True |
| Adam | lr $3\times10^{-3}$, $\beta$=(0.8, 0.99), wd $10^{-4}$ | $1.6 \times 10^{-16}$ | $4.4 \times 10^{-16}$ | True |

*Interpretation.* SGD is bit-for-bit identical because it performs the same floating-point operations in the same order. Adam differs only at the $10^{-16}$ level because PyTorch evaluates the algebraically equal form $\frac{\eta}{1-\beta_1^t}\, m/(\sqrt{v}/\sqrt{1-\beta_2^t}+\epsilon)$, which rounds differently. Both implementations, including bias correction and weight decay, match the reference.

**(b) Trajectories on $f(x, y) = \frac{1}{2}(x^2 + 50y^2)$.** The curvatures are 1 and 50 (condition number $\kappa = 50$), so plain gradient descent is stable only for $\eta < 2/50 = 0.04$. All runs start at $(-9, 2)$ and take 60 steps.

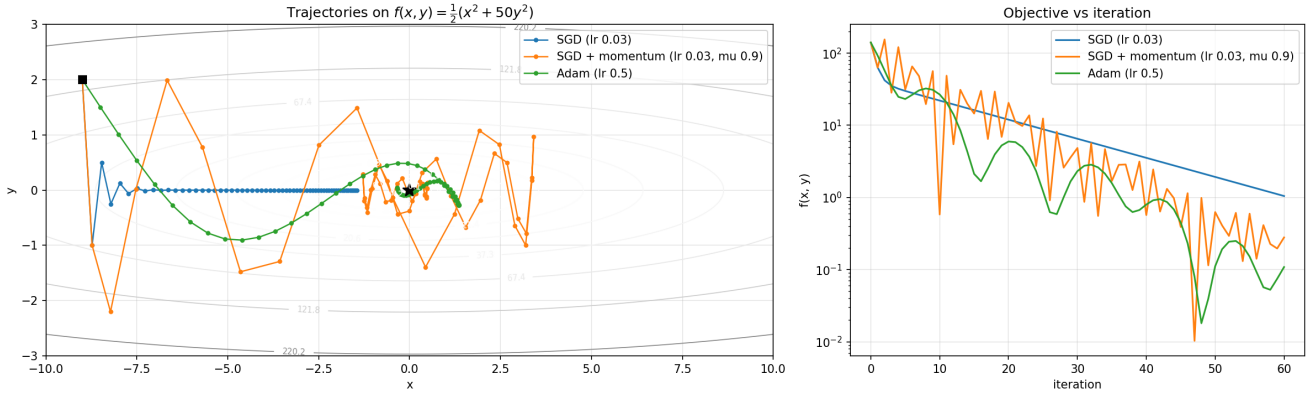



**Figure 3:** Trajectories on the contours of $f$ (left) and $f$ vs iteration (right). Final $f$: SGD 1.05, SGD + momentum 0.28, Adam 0.11.

*Interpretation.*

- **SGD** ($\eta = 0.03$) shows the classic ill-conditioning behaviour. Along $y$ the update factor is $1 - 50\eta = -0.5$, so $y$ flips sign and halves each step, giving the initial zig-zag that dies out within about 10 steps. Along $x$ the factor is $1 - \eta = 0.97$, so $x$ only crawls: $-9 \cdot 0.97^{60} = -1.45$, exactly the observed end point. The learning rate is capped by the *largest* curvature while progress is set by the *smallest*, so convergence slows as $\kappa$ grows.
- **SGD + momentum** ($\eta = 0.03$, $\mu = 0.9$) builds up velocity along the consistently downhill $x$ direction and reaches the minimum region far sooner (final $f$ is 4× lower than SGD's). With $\mu = 0.9$ both directions are under-damped: $(1 + \mu - \eta\lambda)^2 < 4\mu$ holds for $\lambda = 1$ and for $\lambda = 50$. Both modes therefore spiral with modulus $\sqrt{\mu} = 0.95$ per step, which explains the large overshoots in $y$, the non-monotone loss, and $f \approx 140 \cdot 0.9^{60} = 0.25$. The optimum for $\kappa = 50$ would be $\mu^* = \left(\frac{\sqrt{\kappa}-1}{\sqrt{\kappa}+1}\right)^2 \approx 0.57$, so 0.9 is too much momentum for this problem.
- **Adam** ($\eta = 0.5$) normalises each coordinate by its own gradient scale. Early on $\hat{m}/\sqrt{\hat{v}} \approx \text{sign}(g)$, so it moves about 0.5 in *both* coordinates and heads diagonally toward the minimum regardless of the 50:1 curvature ratio. Acting as a diagonal preconditioner, it largely removes the ill-conditioning, which axis-aligned problems like this one are especially suited to. Because the step does not shrink with the gradient, it overshoots in $y$ (the dip to $y \approx -0.9$) and keeps oscillating around the optimum with a constant learning rate. It still reaches the lowest final value ($f = 0.11$).

**(c) Small CNN with each optimizer at three learning rates.** The CNN has 5 conv layers (32-32-64-64-128, ReLU, max-pooling) followed by FC 256 and FC 10, with no batch norm and 0.67M parameters. It was trained on 45k images for 10 epochs (batch 128) at a constant learning rate with no augmentation. The learning rates are one decade apart around each optimizer's usual default.

**Table 3:** Validation accuracy (%) after 10 epochs; best epoch in parentheses. Final training loss on the right.

| | Validation accuracy: final (best) | | | Final training loss | | |
|---|---|---|---|---|---|---|
| Optimizer | low lr | mid lr | high lr | low | mid | high |
| SGD ($\eta = 0.005$ / $0.05$ / $0.5$) | 31.2 (31.5) | **68.1** (68.1) | 10.0 (diverged) | 1.85 | 0.70 | NaN |
| SGD + mom. ($\eta = 0.001$ / $0.01$ / $0.1$) | 47.1 (47.1) | **73.7** (75.1) | 53.1 (57.4) | 1.45 | 0.34 | 1.37 |
| Adam ($\eta = 10^{-4}$ / $10^{-3}$ / $10^{-2}$) | 64.1 (64.1) | **75.7** (76.6) | 9.6 (chance) | 0.98 | 0.19 | 2.30 |




**Figure 4:** Final validation accuracy vs learning rate (left); training loss per epoch and per iteration at each optimizer's best learning rate (middle, right).

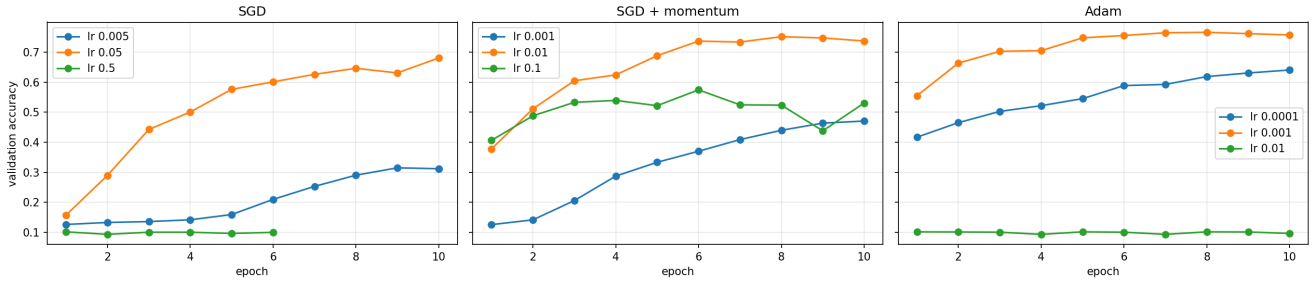



**Figure 5:** Validation accuracy per epoch for every optimizer and learning rate.

*Interpretation.* All three optimizers peak at their mid learning rate. Adam (75.7%) beats SGD + momentum (73.7%), which beats plain SGD (68.1%), and the training-loss curves follow the same order: Adam is the fastest optimiser (0.19 after 10 epochs, against 0.34 and 0.70).

- **Too small a learning rate** is slow rather than harmful. SGD at 0.005 stays on the initial plateau ($L \approx \ln 10 = 2.30$) for about 4 epochs and reaches only 31%. Adam degrades most gracefully on this side: at $10^{-4}$ it still reaches 64%, because its per-parameter normalisation gives small-gradient layers a reasonable step.
- **Too large a learning rate** is catastrophic. Plain SGD at 0.5 diverged to NaN at epoch 6, and Adam at $10^{-2}$ collapsed to a constant prediction (loss stuck at $\ln 10$, chance accuracy). Both are consistent with dead ReLUs or saturation after huge early steps in a network without batch norm. SGD + momentum at 0.1 trains but plateaus and then gets worse (loss rises after epoch 5; validation accuracy drops from 57% to 53%).
- **Effective learning rate.** Momentum multiplies the steady-state step by $1/(1-\mu) = 10$, so SGD + momentum at 0.01 behaves like SGD at 0.1. It is in fact the best SGD variant, and momentum at 0.001 (effective 0.01) lies between SGD's 0.005 and 0.05 results. Momentum mainly reduces gradient noise and oscillation at a given effective step.
- **Sensitivity.** Each optimizer has a useful window of about one decade. Outside it, a factor of 10 costs 15–65 pp or training fails completely, so the learning rate is the most important hyper-parameter to tune. Adam's fast fitting comes with earlier overfitting: its validation accuracy peaks at epoch 8 (76.6%) and then drifts down while the training loss is still falling.

# 3 Task 3 — Regularization and residual connections

**(a) Regularization with 5k training images.** The same small CNN, *with* batch norm after every conv, was trained on the first 5,000 training images (all classes 470–520) and evaluated on the 5k validation split. Every run uses SGD with momentum 0.9, lr 0.05 with cosine decay, batch 128 and 60 epochs. Each regulariser is added to the baseline alone: weight decay $5 \times 10^{-4}$, dropout 0.5 (before both FC layers), flip + crop, cutout $16 \times 16$, and mixup with $\alpha = 1$. The last run combines all of them. Train accuracy is measured on the 5k *clean* images in eval mode, so it is comparable across methods. My cutout and mixup implementations:

```
def cutout(x, size=16):                          # zero a size x size square at a random centre (clipped at border)
    B, C, H, W = x.shape
    cy = torch.randint(0, H, (B, 1, 1), device=x.device); cx = torch.randint(0, W, (B, 1, 1), device=x.device)
    ys = torch.arange(H, device=x.device).view(1, H, 1);  xs = torch.arange(W, device=x.device).view(1, 1, W)
    mask = (ys >= cy - size//2) & (ys < cy + size//2) & (xs >= cx - size//2) & (xs < cx + size//2)
    return x.masked_fill(mask[:, None], 0.0)    # 0 = mean pixel after normalisation
def mixup(x, y, alpha):                          # loss = lam*CE(out, y) + (1-lam)*CE(out, y[p])
    lam = float(np.random.beta(alpha, alpha));  p = torch.randperm(x.size(0), device=x.device)
    return lam * x + (1 - lam) * x[p], y, y[p], lam
```

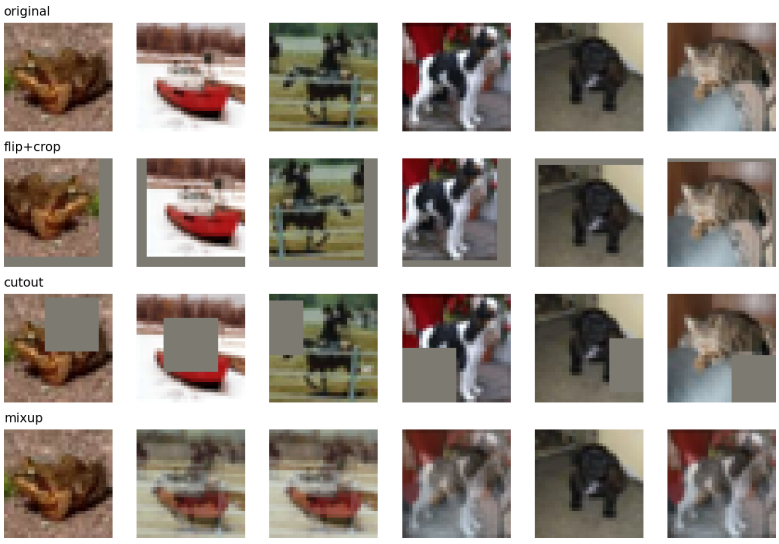



**Figure 6:** The GPU augmentations applied to six training images (grey = zero after normalisation, i.e. the mean pixel).

**Table 4:** Small CNN + BN on 5k training images, after 60 epochs (accuracies in %).

| Configuration | Train acc | Val acc | Gap (train − val) | Best val acc | Final val loss |
|---|---|---|---|---|---|
| baseline | 99.98 | 67.84 | 32.14 | 68.26 | 1.79 |
| weight decay ($5 \times 10^{-4}$) | 100.00 | 68.74 | 31.26 | 68.96 | 1.49 |
| dropout (0.5) | 96.80 | 71.12 | 25.68 | 71.32 | 1.00 |
| flip + crop | 94.20 | **75.32** | 18.88 | **75.54** | **0.83** |
| cutout ($16 \times 16$) | 99.96 | 69.66 | 30.30 | 69.80 | 1.44 |
| mixup ($\alpha = 1$) | 99.94 | 66.44 | 33.50 | 67.16 | 1.03 |
| all combined | 61.60 | 58.80 | **2.80** | 59.36 | 1.19 |

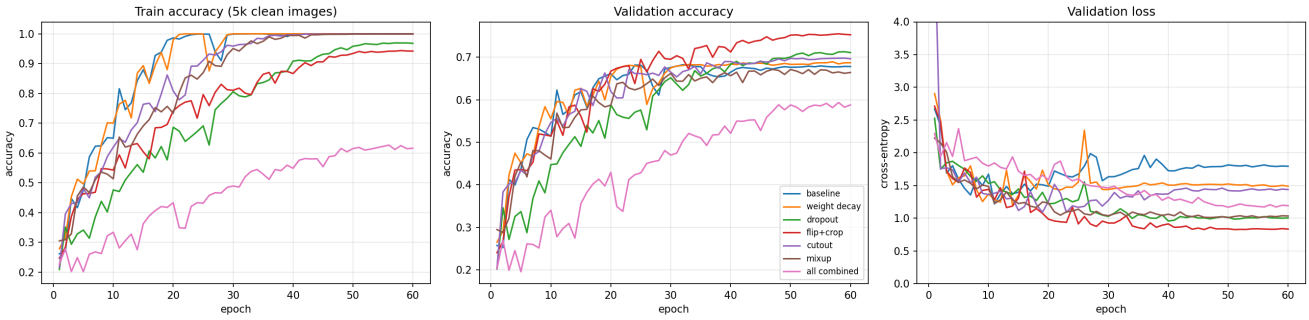



**Figure 7:** Clean train accuracy, validation accuracy and validation loss per epoch for each regulariser.

*Interpretation.*

- **Baseline.** With only 5k images the BN network memorises the training set by epoch ∼24 (99.98% train accuracy) and generalises poorly (67.8%, a 32 pp gap). Its validation loss rises again to 1.79 because it becomes over-confident on the images it gets wrong.
- **Weight decay** barely changes accuracy (+0.9 pp) because, with batch norm, the conv layers are scale-invariant. Decay mainly increases the effective learning rate rather than constraining capacity. It does reduce over-confidence (validation loss 1.49 instead of 1.79).
- **Dropout** is the best explicit regulariser (+3.3 pp, gap 25.7 pp). Because it acts on the 0.53M-parameter FC head, it prevents co-adapted features where most of the parameters are.
- **Flip + crop is by far the most effective single method** (+7.5 pp, 75.3%, gap 18.9 pp, lowest validation loss 0.83). It encodes real invariances of natural images: mirror symmetry and small translations. In effect it multiplies the size of the dataset, so the network never sees the same input twice and cannot memorise.
- **Cutout alone** helps a little (+1.8 pp) but the network still memorises: every image keeps its exact position, so the same pixels can be learned. Cutout is designed to complement flip + crop.
- **Mixup alone** lowers validation accuracy slightly (−1.4 pp) but greatly improves calibration (validation loss 1.03 against 1.79). Mixup makes the targets soft, so it penalises over-confidence. With $\alpha = 1$ and 60 epochs the network still fits the clean images perfectly, and mixup usually needs longer training to pay off in accuracy.
- **All combined** gives the smallest gap (2.8 pp) but the worst accuracy (58.8%), because it *under-fits*: train accuracy is only 61.6% and still rising at epoch 60. Five strong regularisers on a small network within a fixed budget are too much. Regularisers do not add up, and their strengths have to be tuned jointly (e.g. flip +

crop with milder dropout and mixup) or the training has to be longer. A small gap alone is not the goal; what matters is high validation accuracy.

**(b) Plain vs residual networks of depth $6n+2$.** This is the CIFAR ResNet of He et al. (2016): a $3\times3$ conv (16 channels), three stages of $n \in \{1,3,9\}$ basic blocks with 16/32/64 channels (stride 2 at the start of stages 2 and 3), global average pooling and FC. Batch norm follows every conv and initialisation is Kaiming (fan-out). The residual networks use the parameter-free identity shortcut (option A), so the two families have *identical* layers and parameter counts (75k / 270k / 853k) and differ only by the addition. Training: SGD with momentum 0.9, lr 0.1, weight decay $10^{-4}$, cosine decay, flip + crop, 30 epochs on 45k images, mixed precision.

```
class ShortcutA(nn.Module):                    # identity: subsample spatially, zero-pad the new channels
    def forward(self, x):
        if self.stride > 1: x = x[:, :, ::self.stride, ::self.stride]
        if self.extra > 0:  x = F.pad(x, (0, 0, 0, 0, self.extra // 2, self.extra - self.extra // 2))
        return x
class BasicBlock(nn.Module):
    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        if self.shortcut is not None:          # None -> plain network; the ONLY difference
            out = out + self.shortcut(x)
        return F.relu(out)
```

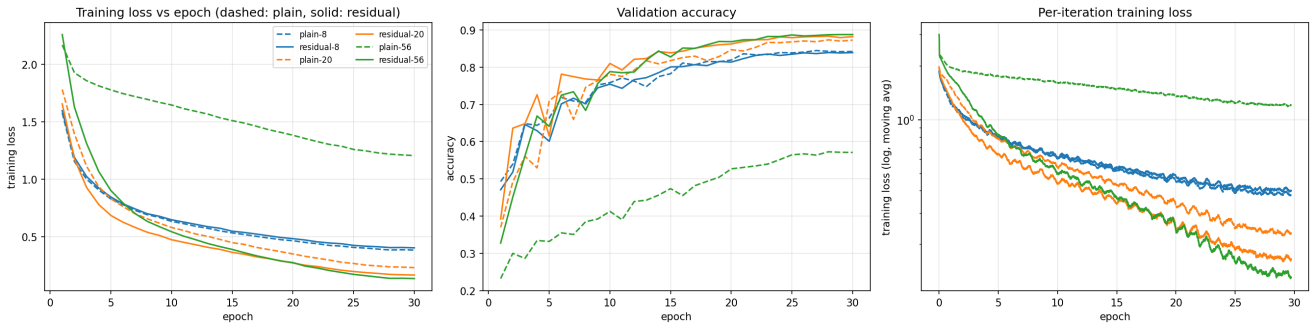



**Figure 8:** Training loss per epoch (left; dashed = plain, solid = residual, colour = depth), validation accuracy (middle), and smoothed per-iteration training loss on a log scale (right).

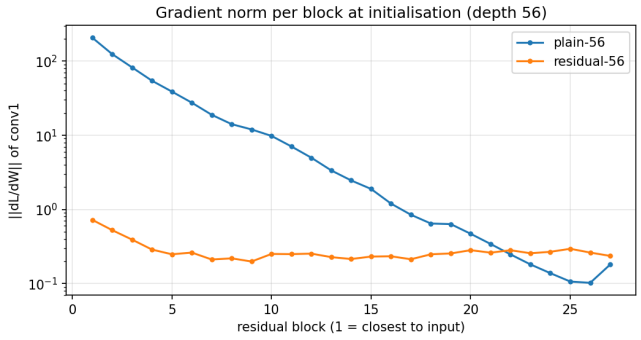



**Figure 9:** $\|\partial L/\partial W\|$ of each block's first conv at initialisation (depth 56, one batch of 256 images).

**Table 5:** After 30 epochs.

| Network | Train loss | Train acc | Val acc |
|---|---|---|---|
| plain-8 | 0.384 | 87.4 | 84.2 |
| residual-8 | 0.403 | 87.1 | 83.9 |
| plain-20 | 0.231 | 92.9 | 87.3 |
| residual-20 | 0.167 | 95.1 | 88.2 |
| plain-56 | **1.205** | 58.6 | 57.1 |
| residual-56 | **0.136** | 96.5 | **88.8** |

*Interpretation.*

- **Depth 8**: plain and residual are indistinguishable (loss 0.384 vs 0.403, accuracy 84.2% vs 83.9%, within noise). A network with 3 blocks is easy to optimise, so the shortcut has nothing to fix.
- **Plain networks degrade with depth.** Plain-20 improves on plain-8, but plain-56 is far *worse* than both: its training loss is 1.205 after 30 epochs, three times plain-8's, and it trains slowly from the first epoch. This is the *degradation problem*, and it is an optimisation failure, not overfitting: train accuracy (58.6%) is as low as validation accuracy (57.1%). A 56-layer plain network could in principle copy plain-20 and set the extra layers to the identity, but SGD cannot find that solution, because an identity map is hard to represent with stacked conv-BN-ReLU layers.
- **Residual networks improve with depth.** The training loss goes 0.403 → 0.167 → 0.136 for depths 8 → 20 → 56, and validation accuracy goes 83.9 → 88.2 → 88.8%. With $x_{l+1} = x_l + F_l(x_l)$, the identity is the default (setting $F_l = 0$) and each block only learns a correction. The gradient $\frac{\partial L}{\partial x_l} = \frac{\partial L}{\partial x_L}\prod_{k\geq l}\left(I + \frac{\partial F_k}{\partial x_k}\right)$ always contains a direct identity path from the loss to every layer.
- **Gradient diagnostic.** At initialisation the per-block gradient norm of plain-56 varies by more than 1000× across depth, from 207 at the first block to 0.1 near the output. In a deep plain network with BN the gradients *explode* toward the input, so no single learning rate suits all layers: the early layers receive huge, noisy updates while the late layers barely move. In residual-56 the norms stay within 0.2–0.7 at every depth. That well-

conditioned gradient flow is why residual-56 trains as easily as residual-20 at the same cost (453 s vs 431 s for plain-56).

# 4 Task 4 — Best CNN with $\leq$ 500k parameters and $\leq$ 20 epochs

**Design.** A wide, shallow ResNet: a $3 \times 3$ conv stem (32 channels), then stages of [2, 2, 1] basic blocks with 32/64/128 channels (stride 2 between stages), global average pooling and FC. It has 12 weight layers and **401,194 parameters**. The wide early stages keep compute at high resolution, and most of the parameters sit in the 128-channel $8 \times 8$ stage. Shortcuts are identity, with $1 \times 1$ projections where the shape changes. The proposed recipe: SGD with Nesterov momentum 0.9, a one-cycle learning rate (peak 0.1, 25% warm-up), weight decay $5 \times 10^{-4}$, label smoothing 0.1, flip + crop + cutout $16 \times 16$, batch 128, mixed precision and 20 epochs (about 2 minutes on a T4). The ablation removes or swaps *one* choice at a time. The model was selected on validation accuracy, and the test set was evaluated once.

**Table 6:** Ablation (accuracies in %).

| Run | Train | Val | Δ val |
|---|---|---|---|
| proposed | 92.2 | 88.64 | — |
| no shortcut (plain) | 91.9 | 88.76 | +0.12 |
| no cutout (flip + crop) | 94.5 | **89.62** | +0.98 |
| no augmentation | 100.0 | 85.68 | −2.96 |
| no weight decay | 90.6 | 86.56 | −2.08 |
| AdamW instead of SGD | 91.4 | 88.30 | −0.34 |
| no label smoothing | 92.1 | 88.36 | −0.28 |

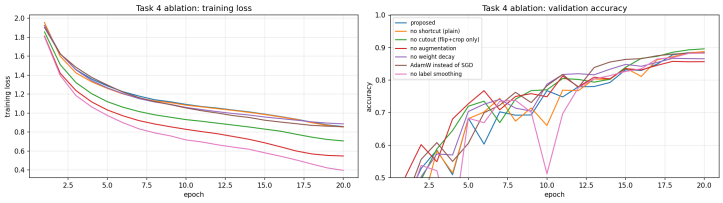



**Figure 10:** Ablation: training loss and validation accuracy.

*Interpretation.*

- **Augmentation matters most.** Without it the network reaches 100% train accuracy but only 85.7% validation accuracy (−3.0 pp): classic overfitting.
- **Weight decay is second** (−2.1 pp without it). With BN, weight decay keeps the weight norms small and thus the effective learning rate high late in training, which acts both as regularisation and as better optimisation. The no-weight-decay run in fact has the *highest* training loss.
- **Cutout hurt within 20 epochs.** Removing it gave the best model (+1.0 pp, train accuracy 94.5% vs 92.2%). Flip + crop + cutout + label smoothing is too much regularisation for this short budget, so the model under-fits, the same effect seen in Task 3(a). Cutout typically pays off only with 100+ epochs.
- **The shortcut made no difference** (+0.1 pp, within noise). The network is only 12 layers deep, consistent with the depth-8 and depth-20 results in Task 3(b). Shortcuts are essential for depth, not for shallow, wide networks.
- **AdamW and label smoothing** changed accuracy by less than 0.4 pp, which is not significant for a single seed. Without label smoothing the training loss is much lower (0.40 vs 0.86) only because the targets are hard, not because the fit is better. That run is also less stable early on (validation accuracy dips at epochs 4 and 10 around the one-cycle peak learning rate).

**Final model and test accuracy.** The selected configuration is the residual network with flip + crop, weight decay $5 \times 10^{-4}$, label smoothing 0.1 and SGD-Nesterov with one-cycle (no cutout), 401,194 parameters, 20 epochs.

**Validation accuracy 89.62%** | **Test accuracy 89.49%** (test loss 0.359, evaluated once)

**Table 7:** Per-class test accuracy (%) of the final model.

| airplane | automobile | bird | cat | deer | dog | frog | horse | ship | truck |
|---|---|---|---|---|---|---|---|---|---|
| 92.2 | 94.4 | 83.6 | **79.1** | 88.8 | 83.9 | 93.5 | 92.4 | 94.2 | 92.8 |

*Interpretation.* Test accuracy (89.49%) is within 0.13 pp of validation accuracy, so choosing among the seven ablation runs on the validation split did not overfit it, and the validation set was a reliable proxy. The errors concentrate on the visually similar animal classes (cat 79.1%, bird 83.6%, dog 83.9%); cat and dog are the classic confusion pair. Rigid man-made objects (car, ship, truck, airplane) all exceed 92%. With more epochs, re-enabling cutout and training on all 50k images would be the next steps.

# Summary

Manual backpropagation matches autograd to machine precision. Activation and initialisation decide whether gradients survive depth, and in the same way identity shortcuts decide whether a 56-layer CNN can be optimised

at all. Among optimisers, the learning rate dominates: every method works only within about one decade. Adam is the fastest and most forgiving at small learning rates; momentum's benefit is mostly an effective-step and noise effect. With scarce data, augmentation that encodes true invariances (flip + crop) beats explicit regularisers, and stacking regularisers under-fits a small, briefly trained network. The final 401k-parameter ResNet reaches 89.49% test accuracy in 20 epochs.

**Acknowledgement of AI use.** Claude (Anthropic) was used to help write and structure the notebook code and to draft this report. I ran all experiments myself on Colab, checked and understood the code, and verified the reported numbers against the notebook outputs. Code and commit history: https://github.com/your-username/en4554-a2.